МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

Оценка комиссии: _____ Рейтинг: _____
Подписи членов комиссии:

_____ _____
(подпись) (фамилия, имя, отчество)

_____ _____
(подпись) (фамилия, имя, отчество)

_____
(дата)

КУРСОВАЯ РАБОТА

по дисциплине **«Методы программирования»**

на тему **Исследование методов сортировки (сортировка пузырьком, гномья сортировка) и поиска (полный перебор (или метод «грубой силы»), двоичный поиск в упорядоченных массивах)**

«К ЗАЩИТЕ»

Доцент кафедры, кандидат технических наук Крючков Алексей Вячеславович
(должность, ученая степень; фамилия, и.о.)

_____
(подпись)

_____
(дата)

ВЫПОЛНИЛ:
Студент группы **КА-22-06**
(номер группы)

Новикова Дарья Дмитриевна
(фамилия, имя, отчество)

_____
(подпись)

_____
(дата)

Москва, 20 23

МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине **«Методы программирования»**

на тему **Исследование методов сортировки (сортировка пузырьком, гномья сортировка) и поиска (полный перебор (или метод «грубой силы»), двоичный поиск в упорядоченных массивах)**

ДАНО студенту **Новиковой Дарье Дмитриевной** группы **КА-22-06**
(фамилия, имя, отчество в дательном падеже) (номер группы)

Содержание работы:

1. Введение
2. 1 Методы сортировки и поиска: теоретическая часть
3. 1.1 Методы сортировки
4. 1.1.1 Сортировка пузырьком: описание и принцип работы
5. 1.1.2 Гномья сортировка: описание и принцип работы
6. 1.1.3 Сравнительный анализ методов сортировки: недостатки и преимущества
7. 1.2 Методы поиска
8. 1.2.1 Полный перебор (или метод «грубой силы»): описание и принцип работы
9. 1.2.2 Двоичный поиск в упорядоченных массивах: описание и принцип работы
10. 1.2.3 Сравнительный анализ методов поиска: недостатки и преимущества
11. 2 Методы сортировки и поиска: практическая часть
12. 2.1 Методы сортировки
13. 2.1.1 Экспериментальное исследование сортировки пузырьком
14. 2.1.2 Экспериментальное исследование гномьей сортировки
15. 2.1.3 Сравнительный анализ и таблицы времени выполнения: выводы
16. 2.2 Методы поиска
17. 2.2.1 Экспериментальное исследование полного перебора (или метода «грубой силы»)
18. 2.2.2 Экспериментальное исследование двоичного поиска в упорядоченных массивах

19. 2.2.3 Сравнительный анализ и таблицы времени выполнения: выводы
20. Заключение
21. Список использованных источников
22. Приложения

Исходные данные для выполнения работы:

Информация об изучаемых методах сортировки и поиска из открытых источников
2. Знания выполняемого курсовую работу языка программирования Python
3.

Рекомендуемая литература:

Кнут Д., Искусство программирования, том.3, Сортировка и поиск: Пер. с англ. – М. ООО «И.Д. Вильямс», 2018. — 832 с.
2. Хоар Ч., Взаимодействующие последовательные процессы: Пер. с англ. – М. Мир, 1989. — 264 с.
3.

Требования к представлению результатов:

✓	Электронная версия
	Бумажный вариант и электронный образ документа

Руководитель: Кандидат технических наук (уч.степень) Доцент кафедры (должность) (подпись) Крючков А.В. (фамилия, имя, отчество)

Задание принял к исполнению: студент (подпись) Новикова Д.Д. (фамилия, имя, отчество)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 МЕТОДЫ СОРТИРОВКИ И ПОИСКА: ТЕОРЕТИЧЕСКАЯ ЧАСТЬ 6

1.1 Методы сортировки 6

1.1.1 Сортировка пузырьком: описание и принцип работы 7

1.1.2 Гномья сортировка: описание и принцип работы 9

1.1.3 Сравнительный анализ методов сортировки: недостатки и преимущества 11

1.2 Методы поиска 12

1.2.1 Полный перебор: описание и принцип работы 13

1.2.2 Двоичный поиск в упорядоченных массивах: описание и принцип работы 15

1.2.3 Сравнительный анализ методов поиска: недостатки и преимущества 17

2 МЕТОДЫ СОРТИРОВКИ И ПОИСКА: ПРАКТИЧЕСКАЯ ЧАСТЬ 18

2.1 Методы сортировки 18

2.1.1 Экспериментальное исследование сортировки пузырьком 18

2.1.2 Экспериментальное исследование гномьей сортировки 20

2.1.3 Сравнительный анализ и таблицы времени выполнения: выводы 22

2.2 Методы поиска 25

2.2.1 Экспериментальное исследование полного перебора 25

2.2.2 Экспериментальное исследование двоичного поиска в упорядоченных массивах 27

2.2.3 Сравнительный анализ и таблицы времени выполнения: выводы 30

ЗАКЛЮЧЕНИЕ 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 36

Приложение А Листинг программы сортировки пузырьком для одномерного массива..... 37

Приложение Б Листинг программы сортировки пузырьком для матриц 38

Приложение В Листинг программы внешней сортировки пузырьком 40

Приложение Г Листинг программы гномьей сортировки для одномерного массива..... 43

Приложение Д Листинг программы гномьей сортировки для матриц 45

Приложение Е Листинг программы внешней гномьей сортировки 48

Приложение Ж Листинг программы поиска полным перебором. Вариант 1. 52

Приложение И Листинг программы поиска полным перебором. Вариант 2 . 54

Приложение К Листинг программы двоичного поиска. Вариант 1 56

Приложение Л Листинг программы двоичного поиска. Вариант 2 59

ВВЕДЕНИЕ

Методы сортировки и поиска представляют большой интерес в области программирования и обработки данных, поскольку они позволяют находить необходимые элементы и упорядочивать их в кратчайшие сроки. В мире современных технологий эффективность алгоритмов решения технических задач нередко зависит от используемого метода сортировки и поиска. Более того, в сфере анализа данных и машинного обучения оптимизация процесса сортировки и поиска является одним из ключевых моментов, влияющих на эффективность работы алгоритмов.

В данной работе изучаются четыре метода работы с данными: сортировка методами «пузырька» и «гномья», а также поиск методами полного перебора и двоичного поиска в упорядоченных массивах.

Актуальность работы заключается в том, что применение оптимальных алгоритмов сортировки и поиска также может улучшить производительность программ и приложений, значительно уменьшить время обработки больших объемов данных, что является ключевым фактором в современном мире информационных технологий.

Цель работы — выявить эффективность каждого из методов и сравнить их между собой. В процессе исследования необходимо проводить замер времени выполнения каждого из методов, что поможет выявить наиболее эффективный метод в различных ситуациях.

В рамках теоретических задач, работа посвящена выявлению основных принципов работы каждого из методов, а также рассмотрению их преимуществ и недостатков.

Практические задачи включают в себя создание нескольких массивов с определенными числами, и применение каждого из методов для их сортировки и поиска определенных элементов. Также будет производиться замер времени выполнения каждого метода для выявления его скорости работы. В итоге исследования нарисуется более полная и точная картина о каждом из методов и их применении в различных ситуациях.

Объект исследования — методы сортировки и поиска данных.

Предмет исследования — сравнение и анализ различных алгоритмов сортировки и поиска для определения наиболее оптимального подхода в зависимости от конкретных задач в области программирования.

Реализация более оптимальных методов сортировки и поиска может улучшить не только производительность программ и приложений, но и общую эффективность работы систем, особенно в области обработки больших объемов данных. Разработчики могут использовать результаты исследования не только для оптимизации текущих алгоритмов, но и для создания новых, более быстрых и эффективных методов, что, в свою очередь, может привести к исследованию новых областей науки и технологий.

1 МЕТОДЫ СОРТИРОВКИ И ПОИСКА: ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Методы сортировки

Многочисленные вычисления и задачи упрощаются благодаря предварительной правильной сортировке информации. Поиск эффективных алгоритмов сортировки доминировал на заре вычислительной техники. Действительно, большая часть ранних исследований в области алгоритмов была сосредоточена на сортировке наборов данных, которые были слишком велики для того, чтобы компьютеры того времени могли хранить их в памяти. Поскольку компьютеры невероятно мощнее и быстрее, чем компьютеры 50-летней давности, размер обрабатываемых наборов данных в настоящее время составляет порядка терабайт информации.

Методы сортировки представляют собой разнообразные алгоритмы, используемые для упорядочивания элементов в определенной последовательности. Они играют ключевую роль в обработке данных и эффективном управлении информацией в различных областях компьютерных наук. В зависимости от конкретных требований задачи выбираются определенные методы, каждый из которых обладает своими уникальными характеристиками по времени выполнения, занимаемой памяти и степени устойчивости к различным входным данным.

1.1.1 Сортировка пузырьком: описание и принцип работы

Сортировка пузырьком — это один из простых алгоритмов сортировки, который можно использовать для упорядочивания элементов в массиве. Назван он так из-за того, что наибольшие (или наименьшие) элементы «всплывают» к своим позициям, как пузырьки воды, пока массив не будет полностью упорядочен.

Описание основных шагов и принципов работы сортировки пузырьком:

1. Проход по массиву:

- Начинаем с начала массива и сравниваем два соседних элемента.
- Если текущий элемент больше следующего, то меняем их местами.

2. Повторение проходов:

- После первого прохода наибольший элемент «всплывёт» в конец массива.
- Повторяем проходы по массиву, исключая последний элемент, так как он уже упорядочен.

3. Завершение процесса:

- Повторяем проходы до тех пор, пока не останется элементов, которые нужно упорядочить.

4. Оптимизация:

- Внутренний цикл проверки и обмена элементов будет выполнен n-1 раз, где n — количество элементов в массиве. Это происходит потому, что наибольший элемент перемещается на свою позицию после первого прохода, на второй после второго прохода и так далее. Таким образом можно сократить число проходов, если на какой-то итерации не было ни одной перестановки элементов.

Схема алгоритма метода сортировки представлена ниже, где arr — некоторый изначально заданный массив, который необходимо отсортировать.

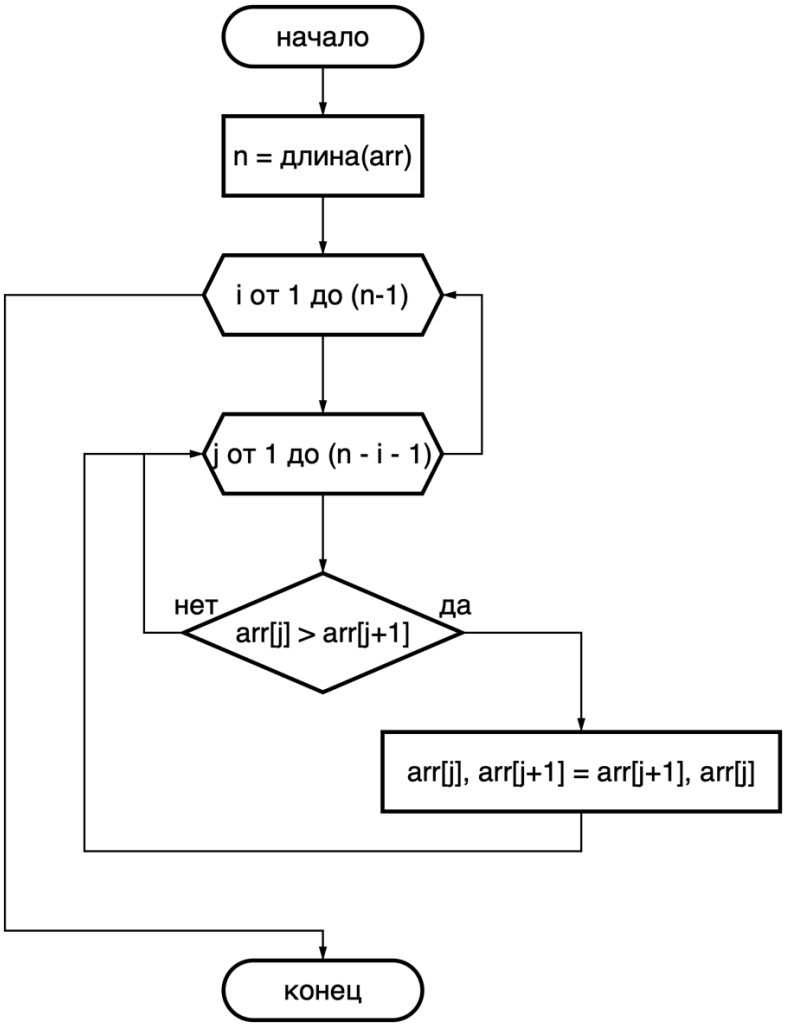

Рисунок 1 — Схема алгоритма сортировки пузырьком

1.1.2 Гномья сортировка: описание и принцип работы

Гномья сортировка является простым алгоритмом сортировки, который можно использовать для упорядочивания элементов в массиве. Этот алгоритм назван в честь гномов, сортирующих вещи в таком порядке, в котором они идут.

Описание основных шагов и принципов работы гномьей сортировки:

1. Инициализация:

- Начинаем с индекса i = 0.

2. Сравнение:

- Сравниваем элемент с индексом i с предыдущим элементом (i-1).

3. Перемещение влево:

- Если элемент с индексом i больше или равен предыдущему элементу, увеличиваем i на 1, чтобы перейти к следующей паре элементов.

- Если элемент с индексом i меньше предыдущего элемента, меняем их местами, уменьшаем ‘i’ на 1 и продолжаем сравнение и перемещение влево до тех пор, пока не достигнем начала массива или не найдем элемент, который больше или равен предыдущему.

4. Повторение процесса:

- После завершения шага 3 у нас будет один элемент, который «всплыл» на свою позицию в отсортированной части массива.

- Увеличиваем i на 1 и повторяем шаги 2 и 3 для следующей пары элементов.

- Процесс продолжается до тех пор, пока i не достигнет конца массива.

5. Завершение:

- Когда i достигнет конца массива, сортировка будет завершена, и элементы массива будут упорядочены.

Схема алгоритма метода сортировки представлена ниже, где arr — некоторый изначально заданный массив, который необходимо отсортировать.

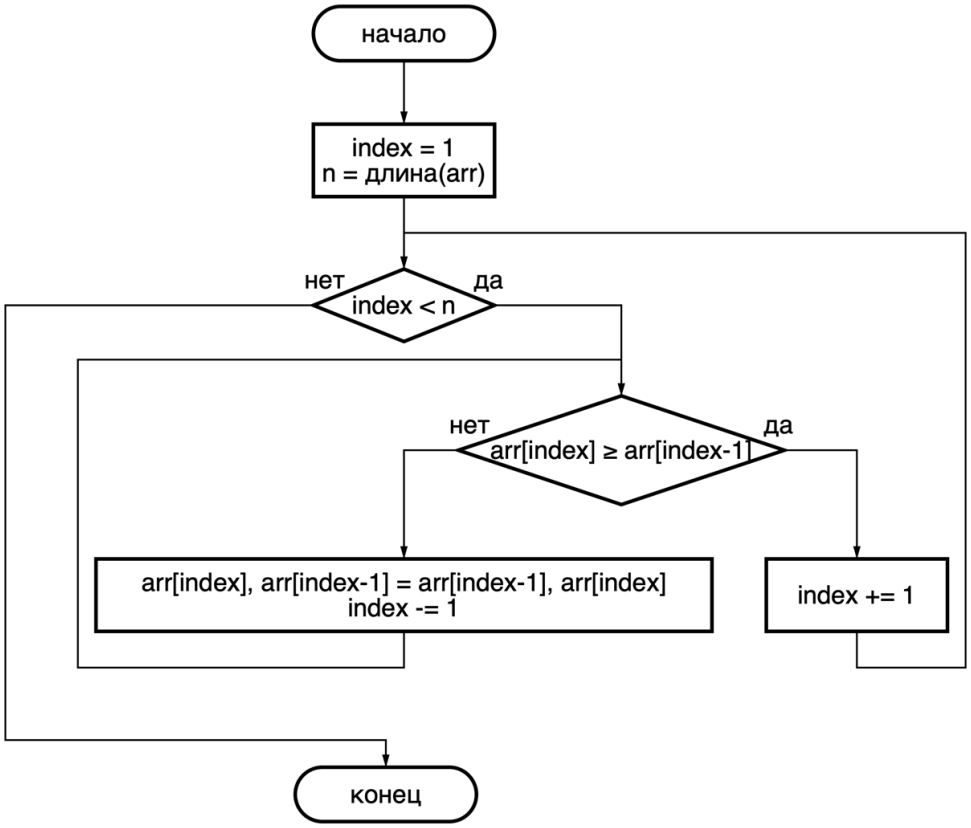

Рисунок 2 — Схема алгоритма гномьей сортировки

1.1.3 Сравнительный анализ методов сортировки: недостатки и преимущества

Сортировка пузырьком:

Преимущества:

- прост в понимании и реализации;
- не требует дополнительной памяти;
- эффективен для небольших массивов или уже частично отсортированных данных.

Недостатки:

- очень низкая эффективность на больших и плохо упорядоченных массивах данных, время выполнения в худшем случае квадратичное и зависит от размера массива.

Гномья сортировка:

Преимущества:

- прост в реализации, но может быть менее интуитивно понятным из-за того, как должны перемещаться элементы;
- не требует дополнительной памяти;
- может быть эффективен для небольших списков.

Недостатки:

- очень низкая эффективность на больших и плохо упорядоченных массивах данных, время выполнения в худшем случае квадратичное и зависит от размера массива.

Таким образом, оба метода просты в реализации и не требуют дополнительной памяти, но неэффективны для больших и плохо упорядоченных массивов данных

1.2 Методы поиска

Получение конкретного фрагмента или фрагментов информации из больших объёмов ранее сохранённых данных — основополагающая операция, называемая поиском, характерная для многих вычислительных задач. Цель поиска — отыскание элементов с ключами, которые соответствуют заданному ключу поиска. Обычно, назначением поиска является получение доступа к информации внутри элемента с целью её обработки.

Поиск используется повсеместно и связан с выполнением множества различных операций. Например, в банке требуется отслеживать информацию о счетах всех клиентов и выполнять поиск в этих записях для подведения баланса и выполнения банковских операций. На авиалинии необходимо отслеживать количество мест на каждом рейсе и выполнять поиск свободных мест, отказа в продаже билетов или внесения каких-либо изменений в списки пассажиров. Ещё один пример — средство поиска в сетевом интерфейсе программы, которое отыскивает в сети все документы, содержащие заданное ключевое слово. Требования, предъявляемые к этим приложениям, в чём-то совпадают (и для банка, и для авиалинии требуются точность и надёжность), а в чём-то различны (банковские данные имеют длительный срок хранения по сравнению с данными остальных упомянутых приложений); тем не менее, во всех случаях требуются эффективные алгоритмы поиска.

1.2.1 Полный перебор: описание и принцип работы

К методам «грубой силы» относят методы решения задачи путем перебора всех возможных вариантов. Сложность полного перебора зависит от размерности пространства всех возможных решений задачи. Если пространство решений очень велико, то полный перебор может не дать результатов в течение месяцев.

Метод грубой силы используется для многих элементарных, но важных алгоритмических задач, таких как: вычисление суммы n чисел, поиск наибольшего элемента в списке, пузырьковая сортировка и т.д.

Идея этого метода заключается в следующем: множество элементов просматривается последовательно в некотором порядке, гарантирующем, что будут просмотрены все элементы множества. Если в ходе просмотра множества будет найден искомый элемент, просмотр прекращается с положительным результатом; если же будет просмотрено все множество, а элемент не будет найден, алгоритм должен выдать отрицательный результат.

Алгоритм поиска полным перебором:

1. Определить область поиска — это может быть список значений, массив или другая структура данных.
2. Определить критерии поиска — это может быть условие, при котором нужное значение будет найдено.
3. Проинициализировать переменную, которая будет хранить информацию о найденном значении.
4. Начать цикл по области поиска.
5. В каждой итерации цикла проверять текущее значение в области поиска при помощи критерия поиска.
6. Если текущее значение соответствует критерию поиска, сохранить это значение в переменной, объявленной в шаге 3.
7. Продолжить цикл до тех пор, пока не будут проверены все значения в области поиска.

8. Проверить, было ли найдено нужное значение.
9. Вернуть найденное значение.

Ниже представлена схема работы алгоритма, где target — искомый элемент в некотором заданном массиве arr.

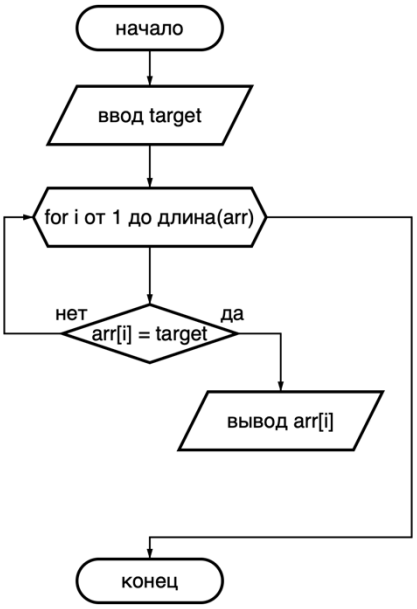

Рисунок 3 — Схема алгоритма полного перебора

1.2.2 Двоичный поиск в упорядоченных массивах: описание и принцип работы

В реализации последовательного поиска на базе массивов общее время поиска на больших наборах элементов можно значительно уменьшить, используя процедуру поиска, основанную на стандартном подходе «разделяй и властвуй». Для этого набор элементов необходимо разделить на две части, определить, к какой из двух частей принадлежит ключ поиска, а затем сосредоточить свои усилия именно на этой части. Рациональный способ разделения наборов элементов на части состоит в поддержке элементов в отсортированном виде с последующим использованием индексов в отсортированном массиве для определения части массива, над которой будет выполняться дальнейшая работа.

Двоичный поиск — это метод поиска, при котором алгоритм ищет элементы в ограниченной области поиска, причём с каждым шагом область поиска делится на две части, используется только для упорядоченных последовательностей, то есть отсортированных по возрастанию или убыванию.

Алгоритм работы двоичного поиска:

1. Массив данных должен быть упорядочен (отсортирован) по возрастанию или убыванию. Алгоритм начинает свою работу с определения границ массива, в которых будет выполняться поиск.
2. На каждом шаге алгоритм выбирает элемент в середине текущего интервала. Сравнивает его с целевым значением:
    - Если значение в середине равно целевому, поиск завершается.
    - Если значение в середине больше целевого, поиск продолжается в левой половине массива.
    - Если значение в середине меньше целевого, поиск продолжается в правой половине массива.

3. Все шаги выполняются до тех пор, пока элемент не будет найден или диапазон поиска не будет сужен до одного элемента.

Ниже приведена схема работы алгоритма поиска, где target — искомый элемент в некотором заданном заранее массиве arr.

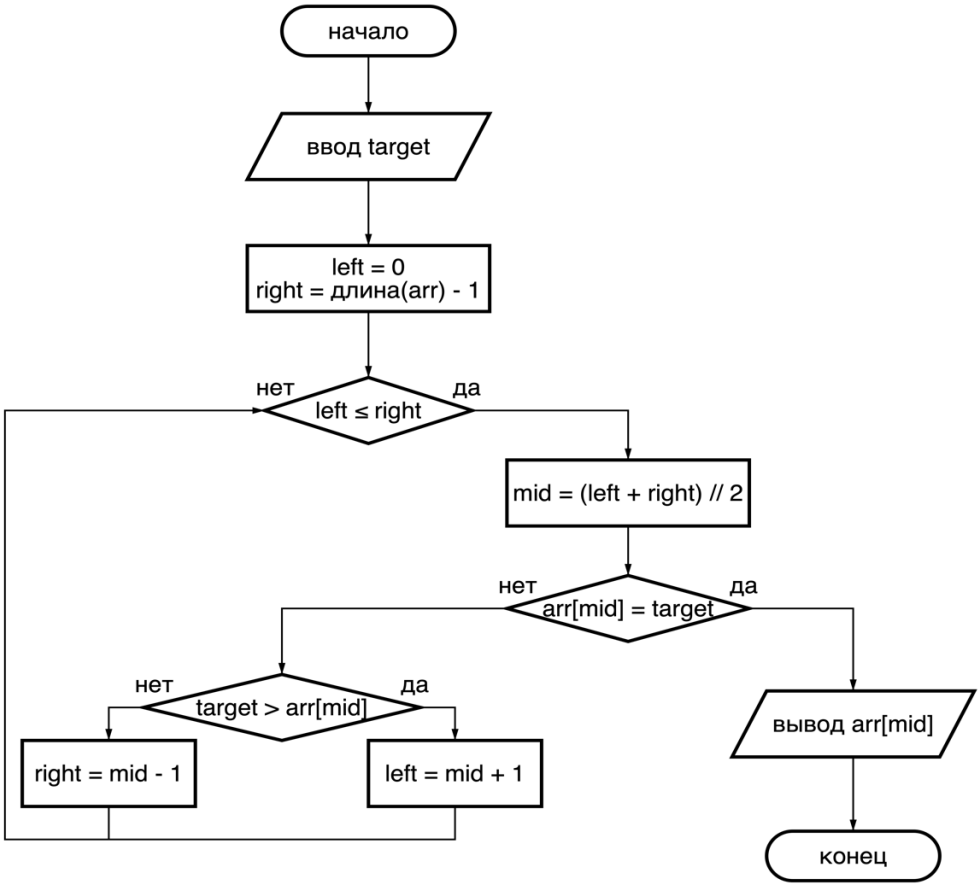

Рисунок 4 — Схема алгоритма двоичного поиска в упорядоченных массивах

1.2.3 Сравнительный анализ методов поиска: недостатки и преимущества

Полный перебор

Преимущества:

- Простота реализации.
- Эффективен для небольших наборов данных или в тех случаях, когда нет информации о структуре массива.

Недостатки:

- Низкая эффективность, потому что в среднем время выполнения поиска полным перебором пропорционально размеру массивах.

Двоичный поиск

Преимущества:

- Высокая эффективность, так как время выполнения двоичного поиска логарифмическое по отношению к размеру массива.

Недостатки:

- Двоичный поиск требует упорядочить массив, что занимает дополнительное время и ресурсы.
- Также для двоичного поиска необходимо, чтобы элементы массива были сравнимыми.

Ввиду своей простоты метод грубой силы удобно использовать в небольших и неупорядоченных массивах. В остальных случаях эффективнее использовать двоичный поиск, несмотря на некоторые ограничения.

2 МЕТОДЫ СОРТИРОВКИ И ПОИСКА: ПРАКТИЧЕСКАЯ ЧАСТЬ

Условия, на которых проводилось тестирование:

– железо: MacBook Pro, чип: Apple M2, Память: 8 ГБ;
– тестирование проводилось на системе mac OS Sonoma 14.0;
– использованная IDE: Visual Studio Code, версия: 1.82.2 (Universal), Python 3.10.11 64-bit.

2.1 Методы сортировки

2.1.1 Экспериментальное исследование сортировки пузырьком

Были написаны программы реализации сортировки пузырьком для наборов данных в виде одномерного массива (см. Приложение А), матрицы (см. приложение Б) и внешней сортировки (см. приложение В). Программы состоят из функции сортировки и основной части. Сама функция сортировки представляет собой следующее:

Листинг программы 1

```
def bubble_sort(arr):
    begin_time = time.time()
    n = len(arr)
    steps = 0
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                steps += 1
    end_time = time.time()
    timer = end_time - begin_time
```

return time, steps, comparisons

Определяется функция сортировки пузырьком. Затем идёт инициализация переменных:

- begin_time — засекаем начальное время выполнения;
- n —длина списка для определения количества элементов;
- steps — переменная для подсчёта шагов;
- comparisons — переменная для подсчёта сравнений.

Внешний цикл (for i in range(n - 1)) проходит по всем элементам до предпоследнего.

Внутренний цикл (for j in range(n - i - 1)) проходит по элементам до непосредственно предыдущего максимального.

Строка (arr[j], arr[j + 1] = arr[j + 1], arr[j]) меняет местами сравниваемые элементы.

Засекаем конечное время выполнения — end_time = time.time(). Вычисляем общее время сортировки — timer = end_time - begin_time.

Возвращается кортеж из времени выполнения, количества шагов и количества сравнений.

Эта программа представляет собой классическую реализацию алгоритма сортировки пузырьком. Ниже представлен пример вывода результата программы для однородного массива размером 5000 элементов:

```
Время, затраченное на выполнение сортировки, составляет:  1.231301  секунд
Количество выполненных перестановок:  6273551
Количество выполненных сравнений:  12497500
```

Рисунок 5 — Пример вывода результата сортировки пузырьком

2.1.2 Экспериментальное исследование гномьей сортировки

Были написаны программы реализации гномьей сортировки для наборов данных в виде одномерного массива (см. Приложение Г), матрицы (см. Приложение Д) и внешней сортировки (см. Приложение Е). Программы состоят из функции сортировки и основной части. Сама функция сортировки представляет собой следующее:

Листинг программы 2

```
def gnome_sort(arr):
    begin_time = time.time()
    index = 0
    n = len(arr)
    steps = 0
    comparisons = 0
    while index < n:
        comparisons += 1
        if index == 0:
            index += 1
        if arr[index] >= arr[index - 1]:
            index += 1
        else:
            arr[index], arr[index - 1] = arr[index - 1], arr[index]
            index -= 1
            steps += 1
    end_time = time.time()
    timer = end_time - begin_time
    return timer, steps, comparisons
```

Определяется функция гномьей сортировки. Затем идёт инициализация переменных:

- begin_time — засекаем начальное время выполнения;

- index — переменная для отслеживания текущего индекса;
- n —длина списка для определения количества элементов;
- steps — переменная для подсчёта шагов;
- comparisons — переменная для подсчёта сравнений.

Цикл самой сортировки:

Увеличиваем счетчик сравнений.

- если текущий индекс указывает на первый элемент, увеличиваем индекс;
- если текущий элемент больше или равен предыдущему, увеличиваем индекс;
- если текущий элемент меньше предыдущего, меняем их местами, уменьшаем индекс и увеличиваем счетчик шагов.

Засекаем конечное время выполнения — end_time = time.time(). Вычисляем общее время сортировки — timer = end_time - begin_time.

Возвращается кортеж из времени выполнения, количества шагов и количества сравнений.

Этот код представляет собой алгоритм гномьей сортировки, который перемещает текущий элемент на своё место, а затем двигается к следующему элементу, поддерживая отсортированный порядок. Ниже представлен пример вывода результата программы для однородного массива размером 5000 элементов:

```
Время, затраченное на выполнение сортировки, составляет:  1.438291  секунд
Количество выполненных перестановок:  6324009
Количество выполненных сравнений:  12653012
```

Рисунок 6 — Пример вывода результата гномьей сортировки

2.1.3 Сравнительный анализ и таблицы времени выполнения: выводы

Порядок ввода и обработки:

– вызываются функции сортировки пузырьком и гномьей сортировки;

– время, затраченное на каждый алгоритм сортировки, измеряется с помощью функции «time.time()»;

– для каждого алгоритма сортировки выводятся затраченное время в секундах, количество перестановок и сравнений.

– Для одномерного массива программа генерирует список, содержащий n случайных целых чисел в диапазоне от -10000 до 10000.

– Для матриц программа генерирует матрицу размером N x N, содержащую случайные целые числа в диапазоне от -10000 до 10000. Сортируются сначала

– Для внешней сортировки программа генерирует два файла, содержащие случайные целые числа в диапазоне от -10000 до 10000. Затем выполняет следующие операции: объединение, пересечение, разность и симметрическая разность. Результаты операций заносятся в новый файл, для которого впоследствии выполняются сортировки.

Итоги работы программ представлены в виде таблиц, содержащих информацию о времени выполнения метода сортировки, количества перестановок и сравнений в ходе работы программы.

Таблица 1 — Сравнение сортировок для одномерных массивов.

Вид сортировки	Количество элементов	Время, с	Количество перестановок	Количество сравнений
Сортировка пузырьком	1000	0.049543	235207	499500
	5000	1.227689	6194191	49995000
	10000	5.035269	25208283	49995000
	20000	19.979448	99617050	199990000
Гномья сортировка	1000	0.063076	251831	504658
	5000	1.444202	6176622	12358235
	10000	5.813290	25026896	50063782
	20000	23.182453	100375702	200771392

Таблица 2 — Сравнение сортировок для матриц.

Вид сортировки	Количество элементов	Время, с	Количество перестановок	Количество сравнений
Сортировка пузырьком	100 x 100	0.083321	510316	990000
	500 x 500	10.310696	62326985	124750000
	1000 x 1000	89.482864	508789335	999000000
	1500 x 1500	300.387361	1678638826	3372750000
Гномья сортировка	100 x 100	0.097327	476669	972337
	500 x 500	12.625381	62413734	125320558
	1000 x 1000	109.182826	500770318	1003526575
	1500 x 1500	367.083795	1677744084	3359964598

Таблица 3 — Сравнение сортировок для файлов. Каждый файл состоит из 3000 элементов.

Вид сортировки	Операция	Время, с	Количество перестановок	Количество сравнений
Сортировка пузырьком	Объединение	1.156524	6217455	13294746
	Пересечение	0.006168	37027	79401
	Разность	0.239681	1270687	2838153
	Сим. разность	0.247128	5829432	11657206
Гномья сортировка	Объединение	1.376878	6317350	12639891
	Пересечение	0.006146	31516	63401
	Разность	0.278677	1291625	2585654
	Сим. разность	0.277381	5791186	11587183

Из таблиц видно, что для любых размеров входных данных гномья сортировка затрачивает больше времени чем сортировка пузырьком. Количество перестановок и сравнений для обеих сортировок в каждом запуске программы почти одинаковое.

При получении на вход большого набора данных обе сортировки обрабатывают их крайне долго. Тем не менее сортировка пузырьком проявила себя немного эффективнее гномьей и оказалась проще в понимании и реализации.

2.2 Методы поиска

2.2.1 Экспериментальное исследование полного перебора

Были написаны программы реализации поиска полным перебором в двух вариантах: исходный файл не меняется (см. Приложение Ж) и из исходного файла случайным образом выбирается 10000 слов (см. Приложение И)

Программы состоят из функции поиска и основной части. Сама функция поиска представляет собой следующее:

Листинг программы 3

```
def search(words,target):
    begin_time = time.time()
    count = 0
    cycle = 0
    for i in words:
        if i == target:
            count += 1
        cycle += 1
    end_time = time.time()
    timer = end_time - begin_time
    return count, timer, cycle
```

Определяется функция поиска полным перебором, которая осуществляет поиск заданного элемента target в списке words. Затем идёт инициализация переменных:

- begin_time — засекаем начальное время выполнения;
- count — переменная для подсчёта частоты элемента target;
- cycle — переменная для подсчёта проходов по циклу.

Проходим по каждому элементу списка. Если элемент равен целевому элементу, увеличиваем счетчик частоты count.

Засекаем конечное время выполнения — end_time = time.time(). Вычисляем общее время сортировки — timer = end_time - begin_time.

Возвращается кортеж из частоты, времени выполнения и количества проходов по циклу для каждого заданного элемента.

Ниже представлен пример вывода результата программы для файла, содержащего 232 слова:

```
Average time:  0.000087
Word            Count      Time Spent      Cycles
==================================================
мышеловка       36         0.000006        232
собака          36         0.000006        232
фальшивой       36         0.000006        232
доберман        26         0.000006        232
йорк            26         0.000005        232
чихуа           26         0.000006        232
кот             12         0.000005        232
кошка           12         0.000006        232
котяра          12         0.000006        232
и               7          0.000005        232
пока            1          0.000005        232
о               1          0.000005        232
привет          1          0.000006        232
```

Рисунок 7 — Пример вывода поиска методом полного перебора

2.2.2 Экспериментальное исследование двоичного поиска в упорядоченных массивах

Были написаны программы реализации двоичного поиска в двух вариантах: исходный файл не меняется (см. Приложение К) и из исходного файла случайным образом выбирается 10000 слов (см. Приложение Л)

Программы состоят из функции поиска и основной части. Сама функция поиска представляет собой следующее:

Листинг программы 4

```
def search(words, target):
    begin_time = time.time()
    count = 0
    cycle = 0
    left = 0
    right = len(words) - 1
    while left <= right:
        cycle += 1
        mid = (left + right) // 2
        if words[mid] == target:
            count += 1
            # Совпадающие слова слева
            i = mid - 1
            while i >= 0 and words[i] == target:
                cycle += 1
                count += 1
                i -= 1
            # Совпадающие слова справа
            i = mid + 1
            while i < len(words) and words[i] == target:
                cycle += 1
```

```
                count += 1
                i += 1
            break
        elif target > words[mid]:
            left = mid + 1
        else:
            right = mid - 1
    end_time = time.time()
    timer = end_time - begin_time
    return count, timer, cycle
```

Определяется функция двоичного поиска, которая осуществляет поиск заданного элемента target в упорядоченном списке words. Затем идёт инициализация переменных:

- begin_time — засекаем начальное время выполнения;
- count — переменная для подсчёта частоты элемента target;
- cycle — переменная для подсчёта проходов по циклу;
- left — индекс левой границы списка;
- right — индекс правой границы списка.

Основной цикл продолжается, пока левая граница не превысит правую. Вычисляется средний индекс списка mid, который сравнивается с заданным элементом. В случае, если средний индекс совпал с заданным элементом, используются два вложенных цикла для поиска совпадающих элементов как слева, так и справа от среднего индекса.

Если целевой элемент больше элемента в середине подсписка, обновляется левая граница. В противном случае обновляется правая граница.

Засекаем конечное время выполнения — end_time = time.time(). Вычисляем общее время сортировки — timer = end_time - begin_time.

Возвращается кортеж из частоты, времени выполнения и количества проходов по циклу для каждого заданного элемента.

Ниже представлен пример вывода результата программы для файла, содержащего 232 слова:

```
Average time:  0.000042
Word            Count      Time Spent      Cycles
==================================================
фальшивой       36         0.000003        37
мышеловка       36         0.000002        36
собака          36         0.000003        38
йорк            26         0.000003        27
доберман        26         0.000002        29
чихуа           26         0.000002        29
котяра          12         0.000001        15
кошка           12         0.000001        14
кот             12         0.000001        16
и               7          0.000001        9
привет          1          0.000001        8
пока            1          0.000001        6
о               1          0.000000        8
darius@Meow-2 ~ %
```

Рисунок 8 — Пример вывода двоичного поиска в упорядоченных массивах

2.2.3 Сравнительный анализ и таблицы времени выполнения: выводы

Вариант 1:

Для данного строкового файла был выполнен частотный анализ появления слов с помощью двоичного поиска в упорядоченных массивах. Программа на вход получает файл, после выполненных действий выводит таблицу, содержащую слово, частота появления этого слова в файле, скорость работы алгоритма для каждого слова, число циклов и всю скорость работы. Таблица отсортирована по убыванию частоты появления слова.

Порядок ввода и обработки:

- программа на вход получает текстовый файл размером 23003 слова, преобразовывает его, убирая знаки препинания и понижая регистр;
- вызываются функции поиска методом полного перебора и двоичного поиска в упорядоченных массивах;
- время, затраченное на каждый алгоритм поиска, измеряется с помощью функции «time.time()»;
- для каждого алгоритма поиска выводятся скорость работы в секундах, частота слов в тексте и количество циклов.

Вариант 2:

Из файла для предыдущего задания с помощью функции «random.sample()» формируется массив, состоящий из случайным образом выбранных 10000 слов. Выполняется проверка уникальности с помощью двоичного поиска в упорядоченных массивах. Выводится таблица, содержащая повторяющиеся элементы, их частота появления в тексте,

скорость работы алгоритма и число обращений к массиву для каждого из повторений. Таблица отсортирована по убыванию частоты появления слов. Порядок ввода и обработки аналогичный варианту 1, но добавляется условие, что частота слов в тексте больше 1.

Итоги работы программ представлены в виде таблиц, содержащих информацию о скорости выполнения метода поиска, частоту слов в тексте, количество циклов программы и среднее время работы программы.

Таблица 4 — Сравнение методов поиска для текстового файла размером 23003 слова

Слово	Частота	Скорость		Количество циклов	
		Полный перебор	Двоичный поиск	Полный перебор	Двоичный поиск
и	1138	0.000657	0.000081	23003	1141
в	712	0.000646	0.000058	23003	715
я	691	0.000648	0.000055	23003	695
не	583	0.000621	0.000039	23003	587
с	426	0.000670	0.000031	23003	427
на	425	0.000631	0.000030	23003	430
…	…	…	…	…	…
дозволите	1	0.000606	0.000002	23003	13
элегией	1	0.000591	0.000001	23003	14

Среднее время работы программы полного перебора составляет 3.865853 секунды.

Среднее время работы программы двоичного поиска составляет 0.015381 секунд.

Таблица 5 — Результаты метода полного перебора для случайного массива из 10000 слов

Слово	Частота	Скорость	Число обращений
и	514	0.000284	10000
в	333	0.000288	10000
я	281	0.000282	10000
не	281	0.000279	10000
что	198	0.000275	10000
с	177	0.000277	10000
на	173	0.000276	10000
…	…	…	…
рука	2	0.000262	10000
частию	2	0.000264	10000

Среднее время работы программы — 2.133804 секунды.

Таблица 6 — Результаты метода двоичного поиска для случайного массива из 10000 слов

Слово	Частота	Скорость	Число обращений
и	517	0.000036	520
в	301	0.000031	304
я	292	0.000030	296
не	258	0.000018	262
на	189	0.000013	194
что	181	0.000017	184
с	174	0.000016	175
…	…	…	…
андрей	2	0.000001	10
решилась	2	0.000002	13

Среднее время работы программы — 0.008265 секунд.

Из таблиц видно, что для любых размеров входных данных метод полного перебора затрачивает больше времени чем двоичный поиск в упорядоченных массивах, потому что он проверяет каждый элемент в массиве. Для двоичного поиска чем меньше частота слова в тексте — тем быстрее выполняется поиск и меньше число циклов. Исходя из этого можно сделать вывод, что двоичный поиск в упорядоченных массивах эффективнее для любых размеров входных данных, поскольку массив данных на каждом шаге разделяется надвое и одна половина сразу отбрасывается.

ЗАКЛЮЧЕНИЕ

Исследование методов сортировки и поиска в массивах является актуальным и важным для программирования и информатики, и позволяет разработчикам выбирать оптимальный подход для достижения наилучшего результата в своей работе.

В процессе исследования были изучены четыре метода работы с массивами — сортировка методами «пузырька» и «гномья», а также поиск методами полного перебора и двоичного поиска в упорядоченных массивах.

Теоретические задачи позволили выявить принципы работы каждого из методов, а также рассмотреть их преимущества и недостатки. Практические задачи позволили нам применить каждый из методов на нескольких массивах и сравнить эффективность их работы.

Результаты по методам сортировки представлены в разделе практической части. Произведено сравнение времени выполнения этих методов, что позволило выявить их эффективность в различных ситуациях. Проанализированы достоинства и недостатки. Аналогичные действия проведены для методов поиска.

Замеры времени выполнения каждого метода показали, что самым быстрым методом поиска является двоичный поиск, а простотой реализации отличается метод сортировки пузырьком.

В ходе исследования все поставленные перед задачи, связанные с анализом методов сортировки и поиска, были успешно решены, и цели были достигнуты. Глубокий анализ алгоритмической сложности различных методов сортировки и поиска подчеркнул, что каждый из них обладает своими уникальными особенностями.

Выделены ключевые моменты, определяющие эффективность этих методов, исходя из особенностей данных, на которых они применяются. Важным выводом является то, что эффективность алгоритмов сильно зависит от характеристик входных данных. Таким образом, оптимальный выбор

метода сортировки или поиска зависит от конкретных требований и свойств данных, с которыми мы имеем дело.

Данные результаты не только подтверждают успешное выполнение поставленных задач, но и предоставляют базу для более глубокого понимания применения методов сортировки и поиска в различных контекстах. Это открывает новые перспективы для оптимизации работы алгоритмов в зависимости от конкретных условий использования.

В целом, тема исследования методов сортировки и поиска остается важным направлением исследований в области информационных технологий и имеет огромный потенциал для будущих разработок и инноваций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ахо Альфред В., Разработка и анализ компьютерных алгоритмов: Пер. с англ. — М. ООО «И.Д. Вильямс», 2021. — 544 с.
2. Барон К., Совершенный алгоритм. Основы: Пер. с англ. – М. Питер, 2022. — 256 с.
3. Бхаргава А., Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих: Пер. с англ. – М. Питер, 2022. — 288 с.
4. Клейнберг Д., Алгоритмы: разработка и применение: Пер. с англ. – М. Питер, 2016. — 800 с.
5. Кнут Д., Искусство программирования, том.3, Сортировка и поиск: Пер. с англ. – М. ООО «И.Д. Вильямс», 2018. — 832 с.
6. Кормен Т., Лейзерсон Ч., Риверст Р., Штайн К. Алгоритмы. Построение и анализ, том 3: Пер. с англ. – М. ООО «И.Д. Вильямс», 2020. — 1328 с.
7. Лутц М. Изучаем Python: Пер. с англ. – М. ООО «И.Д. Вильямс», 2019. — 832 с.
8. Мартин Р. Идеальный программист. Как стать профессионалом разработки ПО: Пер. с англ. – М. Питер, 2022. — 224 с.
9. Мэтиз Э. Изучаем Python: Пер. с англ. – М. Питер, 2020. — 512 с.
10. Панос Л., Алгоритмы для начинающих. Теория и практика для разработчика: Пер. с англ. – М. Бомбора, 2018. — 608 с.
11. Уоррен Г. Алгоритмические трюки для программистов: Пер. с англ. – М. Диалектика, 2019. — 512 с.
12. Хайнеман Д., Алгоритмы. Справочник с примерами на C, C++, Java и Python: Пер. с англ. – М. ООО «И.Д. Вильямс», 2020. — 432 с.
13. Хоар Ч., Взаимодействующие последовательные процессы: Пер. с англ. – М. Мир, 1989. — 264 с.
14. Binstock A. Practical Algorithms for Programmers. Addison-Wesley Professional, 1995. — 592 p.
15. Cormen T., Algorithms Unlocked. MIT Press, 2013. — 240 p.

Приложение А

Листинг программы сортировки пузырьком для одномерного массива

```
import random
import time

def bubble_sort(arr):
    begin_time = time.time()
    n = len(arr)
    steps = 0
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                steps += 1
    end_time = time.time()
    timer = end_time - begin_time
    return time, steps, comparisons

n = 5000
randomlist = [random.randint(-10000, 10000) for _ in range(n)]
time_spent, steps, comparisons = bubble_sort(random list) # Затраченное время, количество шагов перестановки и сравнений
time_spent_sec = "{:.6f}".format(time_spent) # Время в секундах
print("Время, затраченное на выполнение сортировки, составляет: ", time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", steps)
print("Количество выполненных сравнений: ", comparisons)
```

Приложение Б

Листинг программы сортировки пузырьком для матриц

```
import random
import time

def bubble_sort(arr):
    begin_time = time.time()
    n = len(arr)
    steps = 0
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                steps += 1
    end_time = time.time()
    timer = end_time - begin_time
    return timer, steps, comparisons

# Создание матрицы. rows - строки, columns - столбцы
def generate_matrix(rows, columns, min = -10000, max = 10000):
    matrix = [[random.randint(min, max) for _ in range(columns)] for _ in
range(rows)]
    return matrix

# Сортировка матрицы
def b_sort_matrix(matrix):
    total_steps = 0
```

```
    total_time = 0
    total_comparisons = 0
    for row in matrix: # Сортировка всех строк
        time_taken, steps, comparisons = bubble_sort(row)
        total_time += time_taken
        total_steps += steps
        total_comparisons += comparisons

    num_rows = len(matrix) # Количество строк в матрице
    num_columns = len(matrix[0]) # Количество столбцов в матрице
    for col in range(num_columns): # Проход по каждому столбцу
        # Текущий столбец
        current_column = []
        for row in range(num_rows):
            current_column.append(matrix[row][col])

        time_taken, steps, comparisons = bubble_sort(current_column)  #
Сортировка текущего столбца
        total_time += time_taken
        total_steps += steps
        total_comparisons += comparisons
    return total_time, total_steps, total_comparisons

randommatrix = generate_matrix(75, 75)
time_spent, steps, comparisons = b_sort_matrix(randommatrix)
time_spent_sec = "{:.6f}".format(time_spent) # Время в секундах
print("Время, затраченное на выполнение сортировки, составляет: ",
time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", steps)
print("Количество выполненных сравнений: ", comparisons)
```

Приложение В

Листинг программы внешней сортировки пузырьком

```
import random
import time

# Создание файла с случайными числами
def generate_random(filename, n):
    with open(filename, 'w') as f:
        for _ in range(n):
            f.write(str(random.randint(-10000, 10000)) + "\n")
# Чтение файла
def read_f(filename):
    with open(filename, 'r') as f:
        return [int(line.strip()) for line in f.readlines()]

# Запись в файл
def write_f(filename, a):
    with open(filename, 'w') as f:
        for item in a:
            f.write(str(item) + "\n")

# Сортировка пузырьком
def bubble_sort(arr):
    begin_time = time.time()
    n = len(arr)
    steps = 0
    comparisons = 0
    for i in range(n - 1):
        for j in range(n - i - 1):
```

```
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                steps += 1
        end_time = time.time()
        timer = end_time - begin_time
        return arr, timer, steps, comparisons

    generate_random("file1.txt", 2500)
    generate_random("file2.txt", 2500)
    f1 = read_f("file1.txt")
    f2 = read_f("file2.txt")

    # Выполнение операций
    union = list(set(f1).union(set(f2))) # Объединение
    intersection = list(set(f1).intersection(set(f2))) # Пересечение
    difference = list(set(f1).difference(set(f2))) # Разность
    symm_difference = list(set(f1).symmetric_difference(set(f2))) # Симметрическая разность

    sorted_union, u_time_spent, u_steps, u_comparisons = bubble_sort(union)
    u_time_spent_sec = "{:.6f}".format(u_time_spent)
    sorted_intersection, i_time_spent, i_steps, i_comparisons = bubble_sort(intersection)
    i_time_spent_sec = "{:.6f}".format(i_time_spent)
    sorted_difference, d_time_spent, d_steps, d_comparisons = bubble_sort(difference)
    d_time_spent_sec = "{:.6f}".format(d_time_spent)
    sorted_s_difference, s_time_spent, s_steps, s_comparisons = bubble_sort(symm_difference)
```

```
s_time_spent_sec = "{:.6f}".format(d_time_spent)

# Запись результатов в файлы
write_f("sorted_union.txt", sorted_union)
write_f("sorted_intersection.txt", sorted_intersection)
write_f("sorted_difference.txt", sorted_difference)
write_f("sorted_s_difference.txt", sorted_s_difference)

print("Объединение")
print("Время, затраченное на выполнение сортировки, составляет: ", u_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", u_steps)
print("Количество выполненных сравнений: ", u_comparisons)
print("Пересечение")
print("Время, затраченное на выполнение сортировки, составляет: ", i_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", i_steps)
print("Количество выполненных сравнений: ", i_comparisons)
print("Разность")
print("Время, затраченное на выполнение сортировки, составляет: ", d_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", d_steps)
print("Количество выполненных сравнений: ", d_comparisons)
print("Симметрическая разность")
print("Время, затраченное на выполнение сортировки, составляет: ", s_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", s_steps)
print("Количество выполненных сравнений: ", s_comparisons)
```

Приложение Г

Листинг программы гномьей сортировки для одномерного массива

```
import random
import time

# Гномья сортировка
def gnome_sort(arr):
    begin_time = time.time()
    index = 0
    n = len(arr)
    steps = 0
    comparisons = 0
    while index < n:
        comparisons += 1
        if index == 0:
            index += 1
        if arr[index] >= arr[index - 1]:
            index += 1
        else:
            arr[index], arr[index - 1] = arr[index - 1], arr[index]
            index -= 1
            steps += 1
    end_time = time.time()
    timer = end_time - begin_time
    return timer, steps, comparisons

# Создание массива с случайными числами
n = 1000
randomlist = [random.randint(-10000, 10000) for _ in range(n)]
```

```
# Вывод гномьей сортировки
time_spent, steps, comparisons = gnome_sort(randomlist)
time_spent_sec = "{:.6f}".format(time_spent) # Время в секундах
print("Время, затраченное на выполнение сортировки, составляет: ", time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", steps)
print("Количество выполненных сравнений: ", comparisons)
```

Приложение Д

Листинг программы гномьей сортировки для матриц

```
import random
import time

# Гномья сортировка
def gnome_sort(arr):
    begin_time = time.time()
    index = 0
    n = len(arr)
    steps = 0
    comparisons = 0
    while index < n:
        comparisons += 1
        if index == 0:
            index += 1
        if arr[index] >= arr[index - 1]:
            index += 1
        else:
            arr[index], arr[index - 1] = arr[index - 1], arr[index]
            index -= 1
            steps += 1
    end_time = time.time()
    timer = end_time - begin_time
    return timer, steps, comparisons

# Создание матрицы с случайными числами, rows - строки, columns - столбцы
def generate_matrix(rows, columns, min = -10000, max = 10000):
```

```
    matrix = [[random.randint(min, max) for _ in range(columns)] for _ in range(rows)]
    return matrix

# Гномья сортировка матрицы
def g_sort_matrix(matrix):
    total_steps = 0
    total_time = 0
    total_comparisons = 0
    for row in matrix: # Сортировка всех строк
        time_taken, steps, comparisons = gnome_sort(row)
        total_time += time_taken
        total_steps += steps
        total_comparisons += comparisons

    num_rows = len(matrix) # Количество строк в матрице
    num_columns = len(matrix[0]) # Количество столбцов в матрице

    for col_idx in range(num_columns): # Проход по каждому столбцу
        # Извлечение из матрицы текущего столбца
        current_column = []
        for row_idx in range(num_rows):
            current_column.append(matrix[row_idx][col_idx])
        time_taken, steps, comparisons = gnome_sort(current_column) # Сортировка текущего столбца
        total_time += time_taken
        total_steps += steps
        total_comparisons += comparisons
    return total_time, total_steps, total_comparisons
```

```
# Создание матрицы размером n x n
randommatrix = generate_matrix(75, 75)

# Гномья сортировка
time_spent, steps, comparisons = g_sort_matrix(randommatrix)
time_spent_sec = "{:.6f}".format(time_spent) # Время в секундах
print("Время, затраченное на выполнение сортировки, составляет: ", time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", steps)
print("Количество выполненных сравнений: ", comparisons)
```

Приложение Е

Листинг программы внешней гномьей сортировки

```
import random
import time

# Создание файла с случайными числами
def generate_random(filename, n):
    with open(filename, 'w') as f:
        for _ in range(n):
            f.write(str(random.randint(-10000, 10000)) + "\n")
# Чтение файла
def read_f(filename):
    with open(filename, 'r') as f:
        return [int(line.strip()) for line in f.readlines()]

# Запись в файл
def write_f(filename, a):
    with open(filename, 'w') as f:
        for item in a:
            f.write(str(item) + "\n")

# Гномья сортировка
def gnome_sort(arr):
    begin_time = time.time()
    index = 0
    n = len(arr)
    steps = 0
    comparisons = 0
    while index < n:
```

```
            comparisons += 1
            if index == 0:
                index += 1
            if arr[index] >= arr[index - 1]:
                index += 1
            else:
                arr[index], arr[index - 1] = arr[index - 1], arr[index]
                index -= 1
                steps += 1
        end_time = time.time()
        timer = end_time - begin_time
        return arr, timer, steps, comparisons

    # Создание двух файлов со случайными числами
    generate_random("file1.txt", 2500)
    generate_random("file2.txt", 2500)

    # Чтение двух файлов со случайными числами
    f1 = read_f("file1.txt")
    f2 = read_f("file2.txt")

    # Выполнение операций
    union = list(set(f1).union(set(f2))) # Объединение
    intersection = list(set(f1).intersection(set(f2))) # Пересечение
    difference = list(set(f1).difference(set(f2))) # Разность
    symm_difference = list(set(f1).symmetric_difference(set(f2))) #
Симметрическая разность

    # Сортировка и замер времени
    sorted_union, u_time_spent, u_steps, u_comparisons = gnome_sort(union)
```

```
u_time_spent_sec = "{:.6f}".format(u_time_spent)

sorted_intersection, i_time_spent, i_steps, i_comparisons = gnome_sort(intersection)
i_time_spent_sec = "{:.6f}".format(i_time_spent)
sorted_difference, d_time_spent, d_steps, d_comparisons = gnome_sort(difference)
d_time_spent_sec = "{:.6f}".format(d_time_spent)
sorted_s_difference, s_time_spent, s_steps, s_comparisons = gnome_sort(symm_difference)
s_time_spent_sec = "{:.6f}".format(d_time_spent)


# Запись результатов в файлы
write_f("sorted_union.txt", sorted_union)
write_f("sorted_intersection.txt", sorted_intersection)
write_f("sorted_difference.txt", sorted_difference)
write_f("sorted_s_difference.txt", sorted_s_difference)

# Вывод гномьей сортировки
print("Объединение")
print("Время, затраченное на выполнение сортировки, составляет: ", u_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", u_steps)
print("Количество выполненных сравнений: ", u_comparisons)

print("Пересечение")
print("Время, затраченное на выполнение сортировки, составляет: ", i_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", i_steps)
```

```
print("Количество выполненных сравнений: ", i_comparisons)

print("Разность")
print("Время, затраченное на выполнение сортировки, составляет: ", d_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", d_steps)
print("Количество выполненных сравнений: ", d_comparisons)

print("Симметрическая разность")
print("Время, затраченное на выполнение сортировки, составляет: ", s_time_spent_sec, " секунд")
print("Количество выполненных перестановок: ", s_steps)
print("Количество выполненных сравнений: ", s_comparisons)
```

Приложение Ж

Листинг программы поиска полным перебором. Вариант 1

```
import time
# очищение текста, понижение регистра
def text(fname):
    with open(fname, 'r') as file:
        text = file.read()
        words = text.split()
        norm_words = []
        for i in words:
            if i.isalpha():
                i = i.lower()
                norm_words.append(i)
        return norm_words

def search(words,target):
    begin_time = time.time()
    count = 0
    cycle = 0
    for i in words:
        if i == target:
            count += 1
        cycle += 1
    end_time = time.time()
    timer = end_time - begin_time
    return count, timer, cycle

class worddata:
    def __init__(self, word, count, time_spent, cycle):
```

```
        self.word = word
        self.count = count
        self.time_spent = time_spent
        self.cycle = cycle

    fname = '/Users/darius/VS code/ kapitanskaya-dochka.txt'
    words = text(fname)
    target = set(words) # Все уникальные слова в тексте
    word_data_list = []

    begin_time = time.time()
    for i in list(target):
        counts, time_spent, cycle = search(words, i)
        time_spent_sec = "{:.6f}".format(time_spent)
        word_data = worddata(i, counts, time_spent_sec, cycle)
        word_data_list.append(word_data)
    end_time = time.time()
    avg_time = end_time - begin_time
    avg_time_sec = "{:.6f}".format(avg_time)
    print("Average time: ", avg_time_sec)
    sorted_word_data = sorted(word_data_list, key=lambda x: x.count, reverse=True)
    print("{:<15} {:<10} {:<15} {:<15}".format("Word", "Count", "Time Spent", "Cycles"))
    print("="*40)
    for word_data in sorted_word_data:
        print("{:<15} {:<10} {:<15} {:<15}".format(word_data.word, word_data.count, word_data.time_spent, word_data.cycle))
```

Приложение И

Листинг программы поиска полным перебором. Вариант 2

```
import time
import random
# очищение текста, понижение регистра
def text(fname):
    with open(fname, 'r') as file:
        text = file.read()
        words = text.split()
        norm_words = []
        for i in words:
            if i.isalpha():
                i = i.lower()
                norm_words.append(i)
        return norm_words

def search(words,target):
    begin_time = time.time()
    count = 0
    cycle = 0
    for i in words:
        cycle += 1
        if i == target:
            count += 1
    end_time = time.time()
    timer = end_time - begin_time
    return count, timer, cycle

class worddata:
```

```
    def __init__(self, word, count, time_spent, cycle):
        self.word = word
        self.count = count
        self.time_spent = time_spent
        self.cycle = cycle

fname = '/Users/darius/VS code/kapitanskaya-dochka.txt'
words = text(fname)
random_words = random.sample(words, 10000) # Новый текст
target = set(random_words)
word_data_list = []

for i in target:
    counts, time_spent, cycle = search(random_words, i)
    time_spent_sec = "{:.6f}".format(time_spent)
    word_data = worddata(i, counts, time_spent_sec, cycle)
    word_data_list.append(word_data)

sorted_word_data = sorted(word_data_list, key=lambda x: x.count,
reverse=True)
print("{:<15} {:<10} {:<15} {:<15}".format("Word", "Count", "Time
Spent", "Origin Arr Count"))
print("="*60)
for word_data in sorted_word_data:
    if word_data.count > 1:
        print("{:<15} {:<10} {:<15} {:<15}".format(word_data.word,
word_data.count, word_data.time_spent, word_data.cycle))
```

Приложение К

Листинг программы двоичного поиска. Вариант 1

```
import time
# очищение текста, понижение регистра
def text(fname):
    with open(fname, 'r') as file:
        text = file.read()
        words = text.split()
        norm_words = []
        for i in words:
            if i.isalpha():
                i = i.lower()
                norm_words.append(i)
        return norm_words

def search(words, target):
    begin_time = time.time()
    count = 0
    cycle = 0
    left = 0
    right = len(words) - 1
    while left <= right:
        cycle += 1
        mid = (left + right) // 2
        if words[mid] == target:
            count += 1
            # Совпадающие слова слева
            i = mid - 1
            while i >= 0 and words[i] == target:
```

```
            cycle += 1
            count += 1
            i -= 1
        # Совпадающие слова справа
        i = mid + 1
        while i < len(words) and words[i] == target:
            cycle += 1
            count += 1
            i += 1
        break
    elif target > words[mid]:
        left = mid + 1
    else:
        right = mid - 1
end_time = time.time()
timer = end_time - begin_time
return count, timer, cycle

class worddata:
    def __init__(self, word, count, time_spent, cycle):
        self.word = word
        self.count = count
        self.time_spent = time_spent
        self.cycle = cycle

fname = '/Users/darius/VS code/kapitanskaya-dochka.txt'
words = text(fname)
words.sort()
target = set(words) # Все уникальные слова в тексте
word_data_list = []
```

```
begin_time = time.time()
for i in list(target):
    counts, time_spent, cycle = search(words, i)
    time_spent_sec = "{:.6f}".format(time_spent)
    word_data = worddata(i, counts, time_spent_sec, cycle)
    word_data_list.append(word_data)

end_time = time.time()
avg_time = end_time - begin_time
avg_time_sec = "{:.6f}".format(avg_time)
print("Average time: ", avg_time_sec)

sorted_word_data = sorted(word_data_list, key=lambda x: x.count, reverse=True)
print("{:<15} {:<10} {:<15} {:<15}".format("Word", "Count", "Time Spent", "Cycles"))
print("="*50)
for word_data in sorted_word_data:
    print("{:<15} {:<10} {:<15} {:<15}".format(word_data.word, word_data.count, word_data.time_spent, word_data.cycle))
```

Приложение Л

Листинг программы двоичного поиска. Вариант 2

```
import time
import random
# очищение текста, понижение регистра
def text(fname):
    with open(fname, 'r') as file:
        text = file.read()
        words = text.split()
        norm_words = []
        for i in words:
            if i.isalpha():
                i = i.lower()
                norm_words.append(i)
        return norm_words

def search(words, target):
    begin_time = time.time()
    count = 0
    cycle = 0
    left = 0
    right = len(words) - 1
    while left <= right:
        cycle += 1
        mid = (left + right) // 2
        if words[mid] == target:
            count += 1
            # Совпадающие слова слева
            i = mid - 1
```

```
            while i >= 0 and words[i] == target:
                cycle += 1
                count += 1
                i -= 1
            # Совпадающие слова справа
            i = mid + 1
            while i < len(words) and words[i] == target:
                cycle += 1
                count += 1
                i += 1
            break
        elif target > words[mid]:
            left = mid + 1
        else:
            right = mid - 1
    end_time = time.time()
    timer = end_time - begin_time
    return count, timer, cycle

class worddata:
    def __init__(self, word, count, time_spent, cycle):
        self.word = word
        self.count = count
        self.time_spent = time_spent
        self.cycle = cycle

fname = '/Users/darius/VS code/kapitanskaya-dochka.txt'
words = text(fname)
random_words = random.sample(words, 10000) # Новый текст
random_words.sort()
```

```
target = set(random_words)
word_data_list = []

begin_time = time.time()
for i in target:
    counts, time_spent, cycle = search(random_words, i)
    time_spent_sec = "{:.6f}".format(time_spent)
    word_data = worddata(i, counts, time_spent_sec, cycle)
    word_data_list.append(word_data)
end_time = time.time()
avg_time = end_time - begin_time
avg_time_sec = "{:.6f}".format(avg_time)
print("Average time: ", avg_time_sec)

sorted_word_data = sorted(word_data_list, key=lambda x: x.count, reverse=True)
print("{:<15} {:<10} {:<15} {:<15}".format("Word", "Count", "Time Spent", "Origin Arr Count"))
print("="*60)
for word_data in sorted_word_data:
    if word_data.count > 1:
        print("{:<15} {:<10} {:<15} {:<15}".format(word_data.word, word_data.count, word_data.time_spent, word_data.cycle))
```